ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Е.О. Шумова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

«Множественное наследование в языке C++»

по курсу: ОСНОВЫ ПРОГРАММИРОВАНИЯ

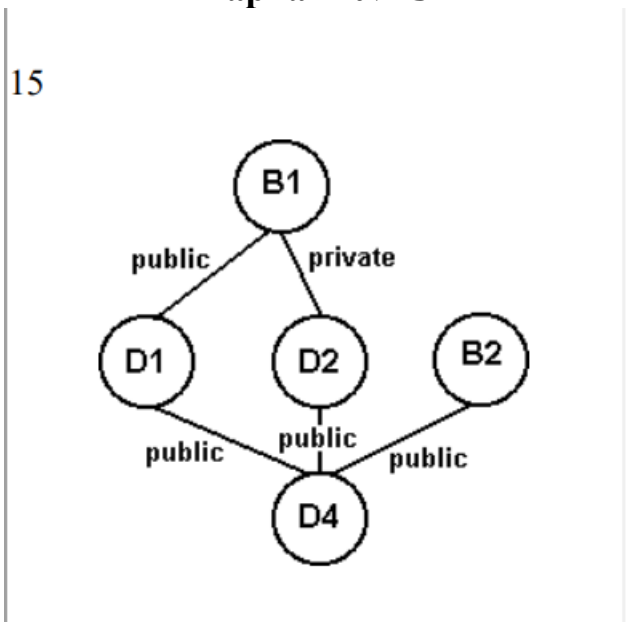
РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Условие с вариантом

Вариант №15



В работе необходимо построить иерархию классов согласно схеме наследования,

приведенной в варианте задания.

Каждый класс должен содержать:

- инициализирующий конструктор с выводом значения, полученного в качестве формального параметра

- функцию show для вывода значений.

- деструктор.

2. Листинг программы

```
#include <iostream>
#include <string>
#include <sstream>

class B1 {
public:
    B1() : data(0) {}  // Конструктор по умолчанию
    B1(int value);
    void show();
    int getData() const;  // Getter
    void setData(int value);  // Setter
    ~B1();

private:
    int data;
};

B1::B1(int value) : data(value) {
```

```
    std::cout << "B1 конструктор: " << data << std::endl;
}

void B1::show() {
    std::cout << "B1: " << data << std::endl;
}

int B1::getData() const {
    return data;
}

void B1::setData(int value) {
    data = value;
}

B1::~B1() {
    std::cout << "B1 деструктор" << std::endl;
}

class B2 {
public:
    B2(int value);
    void show();
    int getData() const;  // Getter
    void setData(int value);  // Setter
    ~B2();

private:
    int data;
};

B2::B2(int value) : data(value) {
    std::cout << "B2 конструктор: " << data << std::endl;
}

void B2::show() {
    std::cout << "B2: " << data << std::endl;
}

int B2::getData() const {
    return data;
}

void B2::setData(int value) {
    data = value;
}

B2::~B2() {
    std::cout << "B2 деструктор" << std::endl;
}

class D1 : virtual public B1 {
public:
    D1(int valueB1, int valueD1);
    void show();
    int getDataD1() const;  // Getter
    void setDataD1(int value);  // Setter
    ~D1();

private:
    int dataD1;
};

D1::D1(int valueB1, int valueD1) : B1(valueB1), dataD1(valueD1) {
    std::cout << "D1 конструктор: " << valueB1 << " " << valueD1 << std::endl;
```

```
}

void D1::show() {
    std::cout << "D1: ";
    B1::show();
    std::cout << "D1: " << dataD1 << std::endl;
}

int D1::getDataD1() const {
    return dataD1;
}

void D1::setDataD1(int value) {
    dataD1 = value;
}

D1::~D1() {
    std::cout << "D1 деструктор" << std::endl;
}

class D2 : virtual private B1 {
public:
    D2(int valueB1, int valueD1, int valueD2);
    void show();
    int getDataD2() const;  // Getter
    void setDataD2(int value);  // Setter
    ~D2();

private:
    int dataD2;
};

D2::D2(int valueB1, int valueD1, int valueD2) : B1(valueB1), dataD2(valueD2) {
    std::cout << "D2 конструктор: " << valueB1 << " " << valueD2 << std::endl;
}

void D2::show() {
    std::cout << "D2: ";
    B1::show();
    std::cout << "D2: " << dataD2 << std::endl;
}

int D2::getDataD2() const {
    return dataD2;
}

void D2::setDataD2(int value) {
    dataD2 = value;
}

D2::~D2() {
    std::cout << "D2 деструктор" << std::endl;
}

class D4 : public D1, public D2, public B2 {
public:
    D4(int valueD1, int valueD2, int valueB2, int valueD4);
    void setB1Value(int value);
    int getB1Value() const;
    void show();
    int getDataD4() const;  // Getter
    void setDataD4(int value);  // Setter
    ~D4();

private:
```

```
    int dataD4;
};

D4::D4(int valueD1, int valueD2, int valueB2, int valueD4)
    : D1(valueD1, valueD1), D2(valueD1, valueD1, valueD2), B2(valueB2), dataD4(valueD4) {
    std::cout << "D4 конструктор: " << valueD1 << " " << valueD2 << " " << valueB2 << " "
<< valueD4 << std::endl;
}

void D4::setB1Value(int value) {
    B1::setData(value);
}

int D4::getB1Value() const {
    return B1::getData();
}

void D4::show() {
    std::cout << "D4: " << dataD4 << std::endl;
    std::cout << "D4: ";
    D1::show();
    D2::show();
    B2::show();
}

int D4::getDataD4() const {
    return dataD4;
}

void D4::setDataD4(int value) {
    dataD4 = value;
}

D4::~D4() {
    std::cout << "D4 деструктор" << std::endl;
}

int main() {
    system("chcp 1251 > nul");
    system("color 71");
    int userValueB1, userValueD1, userValueD2, userValueB2, userValueD4;
    std::string userInput;
    bool validInput = false;

    while (!validInput) {
        std::cout << "Введите значение для B1: ";
        std::cin >> userInput;

        std::istringstream iss(userInput);
        if (iss >> userValueB1) {
            validInput = true;
        }
        else {
            std::cout << "Некорректный ввод. Пожалуйста, введите число." << std::endl;
        }

        // Очистка буфера ввода
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    validInput = false;

    while (!validInput) {
        std::cout << "Введите значение для D1: ";
        std::cin >> userInput;
```

```
    std::istringstream iss(userInput);
    if (iss >> userValueD1) {
        validInput = true;
    }
    else {
        std::cout << "Некорректный ввод. Пожалуйста, введите число." << std::endl;
    }

    // Очистка буфера ввода
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

validInput = false;

while (!validInput) {
    std::cout << "Введите значение для D2: ";
    std::cin >> userInput;

    std::istringstream iss(userInput);
    if (iss >> userValueD2) {
        validInput = true;
    }
    else {
        std::cout << "Некорректный ввод. Пожалуйста, введите число." << std::endl;
    }

    // Очистка буфера ввода
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

validInput = false;

while (!validInput) {
    std::cout << "Введите значение для B2: ";
    std::cin >> userInput;

    std::istringstream iss(userInput);
    if (iss >> userValueB2) {
        validInput = true;
    }
    else {
        std::cout << "Некорректный ввод. Пожалуйста, введите число." << std::endl;
    }

    // Очистка буфера ввода
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

validInput = false;

while (!validInput) {
    std::cout << "Введите значение для D4: ";
    std::cin >> userInput;

    std::istringstream iss(userInput);
    if (iss >> userValueD4) {
        validInput = true;
    }
    else {
        std::cout << "Некорректный ввод. Пожалуйста, введите число." << std::endl;
    }

    // Очистка буфера ввода
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

```
    }

    D4 obj(userValueD1, userValueD2, userValueB2, userValueD4);
    obj.setB1Value(userValueB1);
    obj.show();
    return 0;
}
```

3. Скриншоты с исходными данными и выполнением программы

```
Введите значение для B1: 1
Введите значение для D1: 2
Введите значение для D2: 3
Введите значение для B2: 4
Введите значение для D4: 5
B1 конструктор: 1
D1 конструктор: 1 2
D2 конструктор: 1 3
B2 конструктор: 4
D4 конструктор: 1 2 3 4 5
D4: 5
D4: D1: B1: 1
D1: 2
D2: B1: 1
D2: 3
B2: 4
D4 деструктор
B2 деструктор
D2 деструктор
D1 деструктор
B1 деструктор
```

4. Вывод

Я разработал программу для наглядного примера наследования классов. Она описывает иерархию классов и создает объект класса D4, используя введенные пользователем значения для разных классов B1, B2, D1, D2, D4. Затем она вызывает метод show() для объекта D4 и выводит информацию о конструкторах и деструкторах, которые вызываются в процессе выполнения программы.